

ModernizeProData 개발 매뉴얼

Developer Manual · Enterprise Edition

KS Info System Co., Ltd.
2026

CONTENTS

목차

1. 머리말

2. 시스템 아키텍처

- 2.1 전체 구성도
- 2.2 핵심 설계 원칙
- 2.3 모듈 분리

3. 기술 스택

- 3.1 런타임
- 3.2 프론트엔드
- 3.3 빌드 도구

4. 디렉토리 구조

5. 개발 환경 구축

- 5.1 전제 조건
- 5.2 백엔드
- 5.3 프론트엔드
- 5.4 통합 실행

6. 도메인 모델

- 6.1 핵심 엔티티
- 6.2 주요 enum

7. 메타 데이터베이스 (H2)

- 7.1 마이그레이션 도구
- 7.2 표준 스키마 (요약)
- 7.3 백업

8. 백엔드 구현

- 8.1 패키지 표준
- 8.2 예외 처리 표준
- 8.3 트랜잭션 경계

8.4 보안

9. Spring Batch — 이행 엔진

9.1 Job 구성

9.2 Reader / Processor / Writer

9.3 격리(Quarantine) 처리

9.4 멍등성과 재시작

9.5 진행률 보고

10. JDBC 드라이버 동적 로딩

10.1 요구사항

10.2 구현 방식

10.3 등록 화면

11. 프론트엔드 구현

11.1 라우팅

11.2 상태 관리

11.3 디자인 시스템

11.4 컴포넌트 명명

11.5 폼 검증

12. REST API 명세

12.1 공통 규칙

12.2 엔드포인트 (요약)

12.3 표준 DTO 예시

13. WebSocket 명세

13.1 프로토콜

13.2 토픽

13.3 연결 라이프사이클

14. 인증·인가

14.1 인증 방식

14.2 권한 모델

15. 라이선스 검증

15.1 라이선스 파일 (.lic)

15.2 검증 로직

15.3 시계 되감기 방지

15.4 만료 영향

16. 테스트 전략

16.1 백엔드

16.2 프론트엔드

16.3 회귀

17. 빌드와 패키징

17.1 빌드 산출물

17.2 패키징

17.3 버전 표기

18. 배포와 운영

18.1 시스템 요구사항

18.2 디렉토리 구조 (설치 후)

18.3 systemd 서비스 (Linux)

18.4 모니터링

18.5 백업 / 복구

19. 부록 A. 패키지 구조 표준

20. 부록 B. 명명 규칙

20.1 Java

20.2 TypeScript / React

20.3 SQL

21. 부록 C. 코드 스타일

21.1 Java

21.2 TypeScript

21.3 커밋 메시지

22. 부록 D. 호환성 정책

목차

1. 머리말
2. 시스템 아키텍처
3. 기술 스택
4. 디렉토리 구조
5. 개발 환경 구축
6. 도메인 모델
7. 메타 데이터베이스 (H2)
8. 백엔드 구현
9. Spring Batch — 이행 엔진
10. JDBC 드라이버 동적 로딩
11. 프론트엔드 구현
12. REST API 명세
13. WebSocket 명세
14. 인증·인가
15. 라이선스 검증
16. 테스트 전략
17. 빌드와 패키징
18. 배포와 운영
19. 부록 A. 패키지 구조 표준
20. 부록 B. 명명 규칙
21. 부록 C. 코드 스타일
22. 부록 D. 호환성 정책

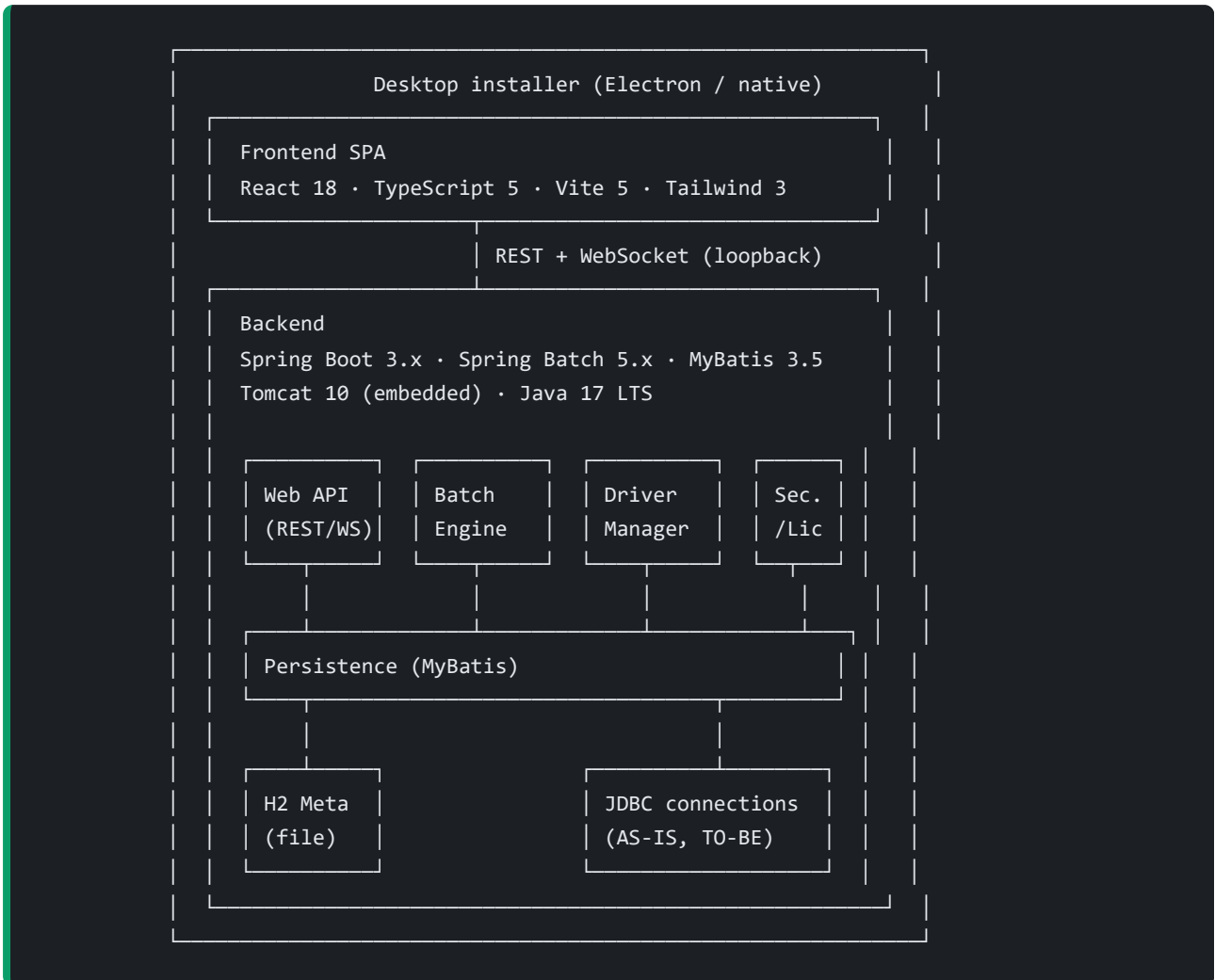
1. 머리말

본 매뉴얼은 ModernizeProData(이하 “본 도구”)의 시스템 구조·구현 방식·확장 절차를 정의합니다. 본 매뉴얼을 따른 구현은 본사 표준 산출물로 간주되며, 본사 검토를 통과해야 릴리즈 후보가 됩니다.

본 도구는 단일 사이트(고객사 데이터센터) 안에 설치되어 운영되는 **온프레미스 데스크탑 설치형 제품**이며, 인터넷 접근이 불가능한 폐쇄망 환경에서도 모든 기능이 동작해야 합니다. 따라서 외부 SaaS 의존성·런타임 모듈 다운로드·자동 업데이트 채널은 채택하지 않습니다.

2. 시스템 아키텍처

2.1 전체 구성도



2.2 핵심 설계 원칙

- **단일 노드 우선** — 본 도구는 한 사이트당 1개 인스턴스를 기본 형태로 합니다. HA·DR 인스턴스를 추가할 수 있으나 동시 실행은 라이선스로 통제됩니다.
- **로컬 우선 통신** — 프론트엔드와 백엔드는 동일 호스트 내 loopback 으로만 통신합니다. 외부 노출이 필요한 경우(External integrations 활성화) 별도 endpoint 를 통해 노출합니다.
- **명시적 사용자 결정** — 자동 추론(컬럼 자동 매핑, 자동 binding 추천 등)을 최소화하고, 모든 매핑은 사용자 결정으로만 활성화됩니다.
- **승인 게이트** — 실행은 반드시 승인된 매핑 스냅샷에 대해서만 가능하며, 매핑 변경·실행은 모두 Audit log 에 기록됩니다.

— 2.3 모듈 분리

백엔드는 다음 모듈로 분리됩니다.

모듈	책임
mpd-core	도메인 모델·공통 유틸리티·예외 정의
mpd-meta	H2 Meta DB 의 마이그레이션·MyBatis 매퍼
mpd-driver	JDBC 드라이버 등록·격리·연결 풀 관리
mpd-batch	Spring Batch Job·Step·Reader·Processor·Writer
mpd-api	REST 컨트롤러·DTO·검증·예외 변환
mpd-ws	WebSocket 채널·이벤트 브로드캐스트
mpd-security	인증·인가·라이선스 검증
mpd-app	Spring Boot 부트스트랩·구성 통합

3. 기술 스택

— 3.1 런타임

항목	버전	비고
Java	17.0.x LTS	Temurin 또는 Azul 권장
Spring Boot	3.2.x	3.3 마이너 업데이트 가능
Spring Batch	5.1.x	
Spring Web (MVC)	Boot 동봉	WebFlux 사용 안 함
Spring WebSocket	Boot 동봉	STOMP over SockJS
MyBatis	3.5.x + mybatis-spring-boot-starter	
H2 Database	2.2.x	file mode (<code>AUTO_SERVER=FALSE</code>)
Tomcat	10.1 (embedded)	

— 3.2 프론트엔드

항목	버전	비고
Node.js	20.x LTS	빌드 시점에만 필요
React	18.x	
TypeScript	5.x	strict 모드
Vite	5.x	번들러
Tailwind CSS	3.x	JIT
react-router	6.x	
zustand 또는 redux-toolkit	최신 안정	상태 관리
axios	1.x	REST 호출
@stomp/stompjs + sockjs-client	최신 안정	WS 클라이언트

— 3.3 빌드 도구

항목	비고
Gradle 8.x	멀티 모듈
pnpm 9.x	프론트엔드 패키지 매니저
jpackage	데스크탑 설치형 패키징

4. 디렉토리 구조

```
ModernizeProData/
├── backend/
│   ├── settings.gradle.kts
│   ├── build.gradle.kts
│   ├── gradle/
│   ├── mpd-app/
│   │   ├── src/main/java/com/ksinfo/mpd/app/
│   │   ├── src/main/resources/
│   │   │   ├── application.yml
│   │   │   └── application-prod.yml
│   │   └── db/migration/ (Flyway 또는 Liquibase 스크립트)
│   └── src/test/java/
│   ├── mpd-core/
│   ├── mpd-meta/
│   ├── mpd-driver/
│   ├── mpd-batch/
│   ├── mpd-api/
│   ├── mpd-ws/
│   └── mpd-security/
├── frontend/
│   ├── package.json
│   ├── vite.config.ts
│   ├── tailwind.config.ts
│   ├── tsconfig.json
│   ├── public/
│   ├── src/
│   │   ├── main.tsx
│   │   ├── App.tsx
│   │   ├── routes/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── stores/
│   │   ├── services/ (API 클라이언트)
│   │   ├── ws/ (WebSocket 클라이언트)
│   │   ├── types/ (DTO 타입 정의)
│   │   ├── utils/
│   │   └── styles/
│   └── tests/
├── installer/
│   ├── electron/
│   └── jpackage/
```

5. 개발 환경 구축

— 5.1 전제 조건

- JDK 17 이 설치되어 있고 `JAVA_HOME` 이 설정되어 있어야 합니다.
- Node.js 20 LTS-pnpm 9 이상이 설치되어 있어야 합니다.
- Git 2.40 이상.

— 5.2 백엔드

```
cd backend
./gradlew clean build           # 전체 빌드
./gradlew :mpd-app:bootRun      # 로컬 실행 (포트 8080)
```

기본 프로파일은 `dev` 입니다. 메타 DB 는 `./data/mpd-meta.mv.db` 에 생성됩니다.

— 5.3 프론트엔드

```
cd frontend
pnpm install
pnpm dev                       # Vite dev server (포트 5173)
```

dev 모드에서는 Vite 가 `/api` 와 `/ws` 를 백엔드(`localhost:8080`)로 프록시합니다(`vite.config.ts`).

— 5.4 통합 실행

`./gradlew :mpd-app:bootRun` 한 번으로 동시 실행되도록 dev 빌드 시 정적 자산을 백엔드 클래스패스에 포함시킬 수 있으나, 개발 단계에서는 위와 같이 분리 실행을 권장합니다.

6. 도메인 모델

6.1 핵심 엔티티

```
Site (사이트)
└─ Project (프로젝트)
    │─ Connection × 2 (asis, tobe)
    │─ DdlImport × 2 (asis, tobe)
    │─ SchemaDiff × N      (TO-BE 테이블별)
    │   │─ Source × N      (binding, JOIN/UNION)
    │   │─ AsisColumn × N
    │   │─ TobeColumn × N
    │   │─ ColumnOverride × N
    │   └─ WhereFilter (선택)
    │─ MappingSnapshot × N  (versions)
    │   └─ Approval (1:1)
    │─ Run × N              (실행 이력)
    │   │─ Stage × N        (extract, transform, load, ...)
    │   └─ Quarantine × N
    └─ AuditLog × N
```

6.2 주요 enum

```
public enum ProjectPhase {
    PLANNING, ANALYSIS, REHEARSAL, SIGN_OFF, CUTOVER, HYPERCARE,
    DONE
}

public enum SnapshotStatus {
    DRAFT, PENDING, APPROVED, REJECTED
}

public enum RunMode {
    REHEARSAL, CUTOVER
}

public enum RunResult {
    RUNNING, OK, WARN, FAILED, ABORTED
}

/**
 * 사용자가 Inspector 에서 명시적으로 선택하는 매핑 전략.
 * RENAME 과 TRANSFORM 을 사용자가 직접 구분하지 않으며,
 * 사용자는 SOURCE_COLUMN 으로 소스 컬럼만 지정하고
 * 시스템이 type delta · SQL 식 유무로 PASSTHROUGH 또는 TRANSFORM 으
    로 변환한다.
 */
public enum MappingStrategy {
    SOURCE_COLUMN, // AS-IS 컬럼 사용 (시스템이
        PASSTHROUGH/TRANSFORM 결정)
    NULL_FILL,     // 명시적 NULL
    DDL_DEFAULT    // DDL DEFAULT 사용
}

/**
 * 컬럼 매핑 그리드의 한 행에 표시되는 룰. 시스템이 결정한 결과 값.
 */
public enum MappingRule {
    PASSTHROUGH, // 소스 컬럼 + 자료형 동일 → 직접 복사
    TRANSFORM,   // 소스 컬럼 + 자료형 다르거나 SQL 변환식 입력
    NULL_FILL,   // NULL 채움
    DDL_DEFAULT, // DDL DEFAULT 적용
    UNMAPPED     // 사용자 결정 대기
}

/**
 * 컬럼 매핑 그리드의 행별 검증 상태.
 */
public enum MappingStatus {
    OK,           // 매핑 완료, 실행 가능
    WARN,         // 실행은 가능하나 검토 권장
    ERROR,        // 실행 불가 (예: NOT NULL 인데 DDL default 도 없음)
    QUEUED        // 결정 대기 중 (UNMAPPED 인 행)
```

```
}

public enum BindingRole {
    PRIMARY, JOIN, UNION
}

public enum ConnectionStatus {
    UNTESTED, OK, FAILED, STALE, TESTING
}
```

7. 메타 데이터베이스 (H2)

— 7.1 마이그레이션 도구

Flyway 또는 Liquibase 중 하나를 채택합니다. 본 매뉴얼은 Flyway 기준으로 기술합니다.

마이그레이션 스크립트는 `mpd-app/src/main/resources/db/migration/` 에
`V<날짜>__<설명>.sql` 형식으로 보관합니다.

7.2 표준 스키마 (요약)

```
-- V20260101__init.sql
CREATE TABLE site (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  code        VARCHAR(40) NOT NULL UNIQUE,
  name        VARCHAR(120) NOT NULL,
  env         VARCHAR(40),
  db_kind     VARCHAR(40),
  db_version  VARCHAR(40),
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE project (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  site_id     BIGINT NOT NULL,
  code        VARCHAR(40) NOT NULL,
  name        VARCHAR(200) NOT NULL,
  domain      VARCHAR(80),
  phase       VARCHAR(20) NOT NULL,
  cutover_at  TIMESTAMP,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_project_site FOREIGN KEY (site_id) REFERENCES
    site(id),
  CONSTRAINT uq_project_site_code UNIQUE (site_id, code)
);

CREATE TABLE connection (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  project_id  BIGINT NOT NULL,
  side        VARCHAR(8) NOT NULL,          -- 'asis' | 'tobe'
  driver_id   VARCHAR(80),                  -- 등록된 driver ref
  jdbc_url    VARCHAR(500),
  username    VARCHAR(120),
  password_enc VARCHAR(500),                -- AES-GCM
  status      VARCHAR(20) NOT NULL DEFAULT 'UNTESTED',
  last_tested_at TIMESTAMP,
  latency_ms  INT,
  CONSTRAINT fk_conn_project FOREIGN KEY (project_id) REFERENCES
    project(id)
);

CREATE TABLE ddl_import (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  project_id  BIGINT NOT NULL,
  side        VARCHAR(8) NOT NULL,
  filename    VARCHAR(300) NOT NULL,
  table_count INT NOT NULL,
  column_count INT NOT NULL,
  imported_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  raw_text    CLOB
);
```

```

CREATE TABLE schema_diff (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  project_id  BIGINT NOT NULL,
  internal_name VARCHAR(120) NOT NULL,
  asis_label  VARCHAR(300),
  tobe_name   VARCHAR(300) NOT NULL,
  where_filter VARCHAR(2000)
);

CREATE TABLE binding_source (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  schema_diff_id BIGINT NOT NULL,
  alias       VARCHAR(40) NOT NULL,
  table_name  VARCHAR(300) NOT NULL,
  role        VARCHAR(10) NOT NULL,
  join_type   VARCHAR(20),
  join_on     VARCHAR(2000),
  ord         INT NOT NULL
);

CREATE TABLE column_override (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  schema_diff_id BIGINT NOT NULL,
  tobe_col_name VARCHAR(300) NOT NULL,
  rule         VARCHAR(20) NOT NULL,
  source_column VARCHAR(300),
  source_alias  VARCHAR(40),
  transform_expr VARCHAR(4000),
  note          VARCHAR(1000),
  edited_by     VARCHAR(80),
  edited_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_override UNIQUE (schema_diff_id, tobe_col_name)
);

CREATE TABLE mapping_snapshot (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  project_id  BIGINT NOT NULL,
  version     VARCHAR(20) NOT NULL,
  status      VARCHAR(20) NOT NULL,
  notes       VARCHAR(2000),
  author      VARCHAR(80) NOT NULL,
  reviewer    VARCHAR(80),
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  approved_at TIMESTAMP,
  approved_by VARCHAR(80),
  rejected_at TIMESTAMP,
  rejected_by VARCHAR(80),
  reject_reason VARCHAR(2000),
  payload_json CLOB NOT NULL,
  CONSTRAINT uq_snapshot_version UNIQUE (project_id, version)
);

```

```

CREATE TABLE migration_run (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  project_id  BIGINT NOT NULL,
  snapshot_id BIGINT NOT NULL,
  mode        VARCHAR(20) NOT NULL,
  result      VARCHAR(20) NOT NULL,
  triggered_by VARCHAR(120) NOT NULL,
  trigger_source VARCHAR(40),
  started_at  TIMESTAMP NOT NULL,
  ended_at    TIMESTAMP,
  rows_total  BIGINT,
  rows_loaded BIGINT,
  rows_quarantined BIGINT,
  CONSTRAINT fk_run_snap FOREIGN KEY (snapshot_id) REFERENCES
    mapping_snapshot(id)
);

CREATE TABLE run_quarantine (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  run_id      BIGINT NOT NULL,
  stage       VARCHAR(40) NOT NULL,
  severity    VARCHAR(20) NOT NULL,
  reason      VARCHAR(500) NOT NULL,
  detail      VARCHAR(2000),
  table_name  VARCHAR(300),
  count_rows  INT NOT NULL,
  first_seen_at TIMESTAMP NOT NULL,
  sample_json CLOB
);

CREATE TABLE audit_log (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  project_id  BIGINT,
  at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  actor       VARCHAR(120) NOT NULL,
  action      VARCHAR(40) NOT NULL,
  target      VARCHAR(300),
  detail      VARCHAR(2000)
);

CREATE INDEX ix_audit_project_at ON audit_log(project_id, at DESC);
CREATE INDEX ix_run_project_started ON migration_run(project_id,
  started_at DESC);

```

— 7.3 백업

H2 file mode 의 데이터 파일(*.mv.db , *.trace.db)을 운영 시간 외에 주기적으로 복사합니다. 운영 절차서의 "Meta DB Backup" 절을 참조하시기 바랍니다.

8. 백엔드 구현

— 8.1 패키지 표준

```
com.ksinfo.mpd
├── core                (공통 - 예외, 도메인 enum, 상수)
├── meta                (메타 DB 매퍼)
│   ├── domain
│   ├── mapper
│   └── repository
├── driver              (JDBC 드라이버 관리)
├── batch               (Spring Batch)
│   ├── job
│   ├── step
│   ├── reader
│   ├── processor
│   └── writer
├── api                 (REST)
│   ├── controller
│   ├── dto
│   ├── mapper          (DTO ↔ 도메인 변환)
│   └── exception
├── ws                  (WebSocket)
│   ├── controller
│   └── event
├── security
└── config              (모든 @Configuration)
```

— 8.2 예외 처리 표준

도메인 예외는 `com.ksinfo.mpd.core.exception` 하위에 정의합니다.

```
public class DomainException extends RuntimeException {
    private final String code;
    private final HttpStatus status;
    // ...
}

// 표준 코드 예시
// PROJECT_NOT_FOUND           404
// SNAPSHOT_NOT_APPROVED       409
// PREFLIGHT_BLOCKED           409
// LICENSE_EXPIRED             402
// DRIVER_NOT_REGISTERED       412
```

`@RestControllerAdvice` 로 변환하여 다음 형태의 응답을 표준화합니다.

```
{
  "code": "SNAPSHOT_NOT_APPROVED",
  "message": "스냅샷 v1.3 은 아직 승인되지 않았습니다.",
  "details": { "snapshotId": 42, "status": "PENDING" },
  "traceId": "a9f3c1..."
}
```

— 8.3 트랜잭션 경계

- 조회는 `readOnly = true` 로 명시
- 쓰기는 서비스 레이어에서 `@Transactional` 1개 경계
- Spring Batch Step 내부의 chunk 트랜잭션은 별도 (자동)

— 8.4 보안

- 메타 DB 비밀번호·외부 DB 비밀번호는 모두 AES-GCM 으로 암호화
- 키는 OS 키링(Windows DPAPI / macOS Keychain) 또는 사이트 KMS 에 보관
- 로그에 비밀번호·자격증명·토큰 출력 금지 (Logback 의 패턴 마스킹 적용)

9. Spring Batch — 이행 엔진

— 9.1 Job 구성

`MigrationJob` 은 다음 Step 으로 구성됩니다.

```
MigrationJob
├─ preflightStep           (검증; 실패 시 Job 차단)
├─ snapshotPinStep        (실행에 사용할 snapshot 고정)
├─ schemaPrepareStep      (TO-BE DDL apply, 인덱스 비활성화 등)
├─ migrationFlow (Flow)
│   ├─ extractStep        (AS-IS → 임시 큐)
│   ├─ transformStep      (룰 적용)
│   ├─ loadStep           (TO-BE → INSERT)
│   └─ validateStep       (FK·NOT NULL 검증, 격리)
├─ schemaFinalizeStep     (인덱스·통계 갱신)
└─ reportStep             (산출물 생성·해시)
```

— 9.2 Reader / Processor / Writer

Reader

`JdbcCursorItemReader` 또는 `JdbcPagingItemReader` 를 사용합니다. 대용량 테이블은 partitioned step 으로 분할하여 파티셔닝 키 기반으로 병렬 실행

행합니다.

Processor

`MappingProcessor` 는 입력 행을 받아 매핑 룰을 적용한 산출 행을 반환합니다. 각 룰의 처리 책임:

```
public interface MappingRuleApplier {
    Object apply(SourceRow src, MappingRow rule, ExecutionContext ctx);
}

// 구현: PassthroughRuleApplier, TransformRuleApplier,
//       NullFillRuleApplier, DdlDefaultRuleApplier
```

Writer

`JdbcBatchItemWriter` 로 INSERT 를 chunk 단위로 commit 합니다. chunk 크기는 사이트 환경에 맞게 외부 설정으로 노출합니다 (기본 1,000).

— 9.3 격리(Quarantine) 처리

`validateStep` 에서 위반이 검출되면 즉시 Job 을 실패시키지 않고 `run_quarantine` 에 적재합니다. 격리 정책:

- 동일 사유 누적이 사이트별 임계치(기본 0.1%) 초과 → Job FAIL
- 그 미만은 Job 은 완료, Run 결과는 `WARN`

— 9.4 멍등성과 재시작

Spring Batch 의 Step Execution 메타데이터를 활용하여 다음을 보장합니다.

- 실패한 Job 의 재실행 시 마지막 커밋 시점부터 재개
- 같은 snapshot+target 으로 두 번 실행 방지 (lock)
- Cutover 모드에서는 `restart-disabled` 정책 적용 (재실행 대신 롤백 절차)

— 9.5 진행률 보고

각 Step 의 chunk 커밋 시 다음 이벤트를 발행합니다.

```
RunProgressEvent {
    runId, stageId, rowsProcessed, rowsTotal,
    throughputRowsPerSec, etaSeconds, errors, warnings
}
```

`mpd-ws` 가 이 이벤트를 구독하여 WebSocket 으로 브로드캐스트합니다.

10. JDBC 드라이버 동적 로딩

— 10.1 요구사항

- 고객사 환경마다 사용하는 DBMS 가 다르므로(Oracle, DB2, Informix, Tiberio, MariaDB, PostgreSQL 등) 본 도구 출하 시점에 모든 드라이버를 포함하지 않습니다.
- 사용자가 *.jar 드라이버를 등록하면 즉시 사용 가능해야 합니다.
- 드라이버끼리의 클래스 충돌(예: 두 Oracle 드라이버 동시 사용)이 발생해도 안전해야 합니다.

— 10.2 구현 방식

각 드라이버는 별도 `URLClassLoader` 인스턴스에 로딩합니다.

```
public final class IsolatedDriver {
    private final String driverId;
    private final ClassLoader loader;
    private final Driver driverInstance;
    // newConnection(jdbcUrl, props) → Connection
}

@Component
public class DriverRegistry {
    private final Map<String, IsolatedDriver> drivers = new
        ConcurrentHashMap<>();

    public void register(String driverId, Path jarPath, String
        driverClassName) {
        URLClassLoader cl = new URLClassLoader(
            new URL[] { jarPath.toUri().toURL() },
            ClassLoader.getSystemClassLoader().getParent() // 부모
            격리
        );
        Driver d = (Driver) Class.forName(driverClassName, true,
            cl)
            .getDeclaredConstructor().newInstance();
        drivers.put(driverId, new IsolatedDriver(driverId, cl, d));
    }
}
```

`DriverManager.registerDriver` 는 사용하지 않습니다(다른 드라이버 라이프사이클과 간섭).

— 10.3 등록 화면

UI: [Solution Settings > Drivers](#) (License 활성화 시 노출).

- 등록 정보: driverId, 표시 이름, jar 파일, driver class FQCN, JDBC URL 템플릿
- 등록 시 자동 검증: 클래스 로딩 가능 여부 + 메타 DB 에 임시 connection 시도

11. 프론트엔드 구현

11.1 라우팅

```

/                                Dashboard
/projects/:id                    (=/) 와 동일
/projects/:id/mapping
/projects/:id/versions
/projects/:id/execution
/projects/:id/artifacts
/projects/:id/logs
/projects/:id/settings/:section?
/sites/:id/overview
/sites/:id/settings

```

11.2 상태 관리

- 서버 상태: TanStack Query (또는 axios + zustand)
- 클라이언트 상태(UI): zustand (사이드바 토글, 선택 상태, 모달, 알림 인박스)

11.3 디자인 시스템

`tailwind.config.ts` 의 토큰을 본 도구의 색·여백 표준으로 정의합니다.

```

// 색상 (light theme 기준)
colors: {
  navy:    '#1f3a5f',    navy50:   '#e7eef8',
  green:   '#0c9e6a',    green50:  '#e2f5ec',
  amber:   '#d97706',    amber50:  '#fef3c7',
  red:     '#c0392b',    red50:    '#fbe8e6',
  text:    '#1c1f23',    text2:    '#3a3f47',
  text3:   '#6b7280',    text4:    '#9ca3af',
  border:  '#e2e6ec',    borderStrong: '#c7ccd4',
  panel:   '#ffffff',    panel2:   '#f7f8fa',
  bg:      '#f3f5f7',
}

```

dark theme 은 `data-theme="dark"` 셀렉터로 동일 키를 오버라이드합니다.

— 11.4 컴포넌트 명명

- 페이지 컴포넌트는 `pages/` 하위에 한 폴더로 (예: `pages/Mapping/`).
- 재사용 컴포넌트는 `components/` 하위 (예: `components/StatusBadge.tsx`).
- 도메인-특화 위젯(매핑 그리드, 인스펙터)은 페이지 폴더 안에 모읍니다.

— 11.5 폼 검증

react-hook-form + zod 스키마로 모든 입력 폼을 통합합니다. 서버 측 검증 오류는 `@RestControllerAdvice` 에서 동일 zod 호환 형식으로 변환하여 폼에 표시합니다.

12. REST API 명세

— 12.1 공통 규칙

- Base path: `/api/v1`
- 인증: `Authorization: Bearer <jwt>` (Solution Settings 에서 외부 노출 활성 시) 또는 세션 쿠키(로컬)
- 응답 형식: JSON
- 페이지징: `?page=0&size=50` (Spring Data 호환)

— 12.2 엔드포인트 (요약)

사이트 / 프로젝트

```
GET    /sites
GET    /sites/{siteId}
GET    /sites/{siteId}/projects
POST   /sites/{siteId}/projects      Body: ProjectCreateRequest
GET    /projects/{projectId}
PATCH /projects/{projectId}      Body: ProjectPatch
DELETE /projects/{projectId}
POST   /projects/{projectId}:duplicate
```

DDL & 연결

```
POST /projects/{id}/connections/{side}      Body: ConnectionRequest
POST /projects/{id}/connections/{side}:test → ConnectionResult
POST /projects/{id}/ddl/{side}              Body: multipart/form-data
DELETE /projects/{id}/ddl/{side}
```

매핑

```
GET /projects/{id}/inventory      → AsisInventory + TobeInventory
GET /projects/{id}/schema-diff    → SchemaDiff[]
GET /projects/{id}/schema-diff/{internalName}/mappings
PUT /projects/{id}/schema-diff/{internalName}/bindings
                                   Body: BindingsUpdate
PUT /projects/{id}/schema-diff/{internalName}/where-filter
                                   Body: { whereFilter: string }
PUT /projects/{id}/schema-diff/{internalName}/columns/{col}
                                   Body: ColumnOverride
DELETE /projects/{id}/schema-diff/{internalName}/columns/{col}
```

스냅샷 / 승인

```
GET /projects/{id}/snapshots
POST /projects/{id}/snapshots      Body: SnapshotCreate
GET /snapshots/{snapshotId}
POST /snapshots/{snapshotId}:approve
POST /snapshots/{snapshotId}:reject Body: { reason: string }
```

실행

```
GET /projects/{id}/preflight      → PreflightCheck[]
POST /projects/{id}/runs           Body: RunCreateRequest
GET /runs/{runId}
POST /runs/{runId}:pause
POST /runs/{runId}:resume
POST /runs/{runId}:abort
GET /runs/{runId}/quarantine
GET /projects/{id}/runs            (history)
```

산출물

```
GET /runs/{runId}/artifacts/{kind}      kind: mapping-yaml | bindings-yaml |
                                           Accept: text/plain | application/oct
```

Audit / 알림

```
GET    /projects/{id}/audit-log      ?since=...&limit=...
GET    /notifications                ?unread=true
POST   /notifications/{id}:read
POST   /notifications:read-all
DELETE /notifications
```

솔루션 설정 / 라이선스

```
GET    /solution/settings
PUT    /solution/settings
GET    /solution/license
POST   /solution/license:upload      Body: multipart .lic
GET    /solution/drivers
POST   /solution/drivers           Body: multipart .jar + meta
DELETE /solution/drivers/{driverId}
```


— 12.3 표준 DTO 예시

```
/**
 * 사용자가 Inspector 에서 저장한 매핑 결정. 입력 측 모델로 strategy
 *   는
 * MappingStrategy enum 의 값(SOURCE_COLUMN | NULL_FILL |
 *   DDL_DEFAULT) 이며,
 * 그외 표시용 MappingRule(PASSTHROUGH/TRANSFORM/...)은 시스템이 별
 *   도로 derive 한다.
 */
public record ColumnOverride(
    MappingStrategy strategy,      // 사용자 입력 전략
    String sourceColumn,          // strategy = SOURCE_COLUMN 일
        때만 필수
    String sourceAlias,           // 동일 (binding 의 alias)
    String transformExpr,         // strategy = SOURCE_COLUMN,
        optional SQL 식
    String note,
    String editedBy,
    Instant editedAt
) {}

public record PreflightCheck(
    String id,
    String title,
    String status,                // "pass" | "warn" | "fail" |
        "skip"
    String detail,
    FixHint fix
) {
    public record FixHint(String tab, String section, FixTarget
        target) {}
    public record FixTarget(String side, String tobeName, String
        internalName, String colName) {}
}
```

13. WebSocket 명세

— 13.1 프로토콜

STOMP over SockJS. Endpoint: `/ws` .

— 13.2 토픽

Destination	페이로드	설명
/topic/runs/{runId}/progress	RunProgressEvent	chunk 커밋마다 발행
/topic/runs/{runId}/log	LogLineEvent	실시간 로그 라인
/topic/runs/{runId}/quarantine	QuarantineEvent	격리 추가 시
/topic/projects/{projectId}/notifications	NotificationEvent	인앱 알림 푸시
/topic/system/health	HealthEvent	DB·라이선스·연결 상태

— 13.3 연결 라이프사이클

- 클라이언트는 페이지 로드 시 `/ws` 로 연결
- 활성 Run 페이지 진입 시 해당 토픽 subscribe, 이탈 시 unsubscribe
- 60초 ping/pong, 끊김 감지 시 지수 백오프 재연결

14. 인증·인가

— 14.1 인증 방식

- 로컬 모드(기본) — 단일 사용자(설치 시 정의)·세션 쿠키
- 확장 모드(엔터프라이즈) — Spring Security + JWT, 사이트 LDAP/AD 연동

— 14.2 권한 모델

Owner — 모든 동작
Maintainer — 매핑·실행 가능, 프로젝트 삭제 불가
Reviewer — 스냅샷 승인 가능, 매핑 편집 불가
Viewer — 읽기 전용

API 측 인가는 `@PreAuthorize` 표현식으로 명시합니다.

```
@PreAuthorize("hasAnyRole('OWNER','MAINTAINER')")
public ResponseEntity<Void> updateBinding(...) { ... }

@PreAuthorize("hasRole('REVIEWER') or hasRole('OWNER')")
public ResponseEntity<Snapshot> approve(...) { ... }
```

15. 라이선스 검증

— 15.1 라이선스 파일 (.lic)

페이로드 + Ed25519 서명을 base64 로 묶은 단일 파일.

```
{
  "v": 1,
  "licenseId": "MPD-2026-0421-KDB",
  "customer": "KDB Bank",
  "siteId": "s1",
  "edition": "enterprise",
  "features": ["multi-site", "external-trigger"],
  "issuedAt": "2026-04-21",
  "expiresAt": "2027-04-21",
  "graceDays": 14
}
```

`signature` 는 위 페이로드의 JCS 정규화 + SHA-256 + Ed25519 서명입니다.

— 15.2 검증 로직

```
@Component
public class LicenseVerifier {
    private final byte[] publicKey; // 빌드 타임 임베드

    public LicenseStatus verify(byte[] licFile) {
        LicensePayload payload = parseAndVerifySignature(licFile,
            publicKey);
        if (!siteIdMatches(payload.siteId())) return
            LicenseStatus.SITE_MISMATCH;
        Instant now = systemClock.instant();
        if (now.isBefore(payload.issuedAt())) return
            LicenseStatus.NOT_YET_VALID;
        if (now.isAfter(payload.expiresAt().plus(graceDays)))
            return LicenseStatus.EXPIRED;
        if (now.isAfter(payload.expiresAt())) return
            LicenseStatus.IN_GRACE;
        return LicenseStatus.ACTIVE;
    }
}
```

— 15.3 시계 되감기 방지

마지막 검증 시각을 AES-GCM 으로 봉인하여 `data/license-state.bin` 에 저장합니다. 검증 시 현재 시각이 봉인된 last-seen 보다 이전이면

`CLOCK_TAMPERED` 로 처리하고 라이선스를 비활성화합니다.

— 15.4 만료 영향

상태	도구 동작
ACTIVE	모든 기능 사용 가능
IN_GRACE	모든 기능 사용 가능 + 화면 상단 빨간 배너
EXPIRED	매핑 편집·실행 차단, 조회·내보내기만 가능
SITE_MISMATCH / CLOCK_TAMPERED	모든 기능 차단, 사이트 관리자 안내 화면

16. 테스트 전략

— 16.1 백엔드

- **단위** — JUnit 5 + AssertJ + Mockito. 도메인 로직(룰 적용기, 매핑 생성기, 서명 검증) 100% 커버.
- **MyBatis 매퍼** — H2 in-memory 로 매퍼 단위 테스트.
- **Batch** — `@SpringBatchTest` 로 Job·Step 단위 검증.
- **API** — `@WebMvcTest` 또는 `@SpringBootTest(webEnvironment=RANDOM_PORT)` 와 RestAssured.
- **이행 시나리오 통합** — Testcontainers 로 PostgreSQL·Oracle XE 컨테이너 띄워 end-to-end 시나리오 실행.

— 16.2 프론트엔드

- **단위** — Vitest + Testing Library.
- **E2E** — Playwright. 핵심 시나리오: 로그인 → 프로젝트 생성 → DDL 도입 → binding → 매핑 → 스냅샷 → 승인 → run.

— 16.3 회귀

- 모든 PR 은 단위 + 매퍼 + API 테스트가 통과해야 머지 가능
- 메이저 릴리즈 이전에 통합 시나리오 + E2E 실행 및 회귀 보고서 첨부

17. 빌드와 패키징

— 17.1 빌드 산출물

- `mpd-app/build/libs/mpd-app-{version}.jar` (Spring Boot fat jar)
- `frontend/dist/` (정적 자산, 본 자산을 fat jar 의 `static/` 으로 포함)

— 17.2 패키징

본사 표준은 두 형태를 동시 제공합니다.

A. Standalone fat jar

```
java -jar mpd-app-1.0.0.jar
```

설치 호스트의 OS 가 임의여도 동작.

B. 데스크탑 설치형 (jpackage)

```
jpackage --type msi --name ModernizeProData \  
  --input dist/ \  
  --main-jar mpd-app-1.0.0.jar \  
  --main-class org.springframework.boot.loader.launch.JarLauncher \  
  --runtime-image build/jdk-17-runtime \  
  --icon installer/icons/mpd.ico \  
  --vendor "KS Info System Co., Ltd." \  
  --app-version 1.0.0
```

Windows 는 `.msi` , macOS 는 `.dmg` , Linux 는 `.deb` / `.rpm` 으로 산출합니다.

— 17.3 버전 표기

`major.minor.patch` (Semantic Versioning). About 모달의 build 식별자는 다음 형식으로 자동 주입합니다.

```
v1.0.0 · build 2026.04.18-a9f3c1
```

`a9f3c1` 은 Git 단축 SHA.

18. 배포와 운영

— 18.1 시스템 요구사항

항목	최소	권장
CPU	4 cores	8 cores 이상
RAM	8 GB	16 GB 이상
Disk	50 GB (Meta + 산출물)	200 GB SSD
Java	17.0.x LTS	17.0.10 이상
OS	Windows Server 2019 / RHEL 8 / Ubuntu 20.04	동일 최신 패치

— 18.2 디렉토리 구조 (설치 후)

```
<install>/
├─ bin/                실행 스크립트
├─ lib/                fat jar
├─ runtime/           동봉 JRE
├─ data/
│   ├── mpd-meta.mv.db  H2 메타
│   ├── license-state.bin  봉인된 라이선스 상태
│   ├── drivers/        등록된 JDBC 드라이버
│   └─ runs/            Run 산출물
├─ config/
│   ├── application.yml
│   └─ public-key.pem    라이선스 검증용 공개키 (사실상 빌드 임베드)
└─ logs/
    ├── app.log
    └─ audit.log
```

— 18.3 systemd 서비스 (Linux)

```
[Unit]
Description=ModernizeProData
After=network.target

[Service]
Type=simple
User=mpd
WorkingDirectory=/opt/modernizeprodata
ExecStart=/opt/modernizeprodata/bin/mpd
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

— 18.4 모니터링

- `/api/v1/health` 엔드포인트 (Spring Actuator)
- 로그는 logback-spring.xml 의 RollingFileAppender 로 분리:
 - `logs/app.log` — 애플리케이션 로그 (DEBUG/INFO/WARN/ERROR)
 - `logs/audit.log` — Audit log (변경 불가)
 - `logs/access.log` — HTTP 액세스
- Syslog 전달은 [Solution Settings > External integrations](#) 에서 활성화

— 18.5 백업 / 복구

운영 절차서의 "백업·복구" 장 참조. 핵심 항목:

- Meta DB 일일 백업 (cron + `cp` 또는 H2 의 `BACKUP TO`)
- 등록 드라이버 디렉토리 함께 백업
- 라이선스 파일은 본사 보관본을 정본으로 함

19. 부록 A. 패키지 구조 표준

본 도구의 백엔드 패키지는 `com.ksinfo.mpd` 를 루트로 합니다. 모든 신규 클래스는 위의 11.1 디렉토리 트리를 준수해야 합니다.

신규 모듈 추가 시:

1. `settings.gradle.kts` 에 모듈 등록
2. 모듈 `build.gradle.kts` 의 dependencies 에 의존 모듈만 명시 (역방향 의존 금지)

3. 모듈 단위 `package-info.java` 에 책임 한 줄 기재
-

20. 부록 B. 명명 규칙

— 20.1 Java

- 클래스: PascalCase, 약어는 첫 글자만 대문자 (`JdbcUrl` 가 아니라 `JdbcUrl`)
- 메서드: camelCase, 동사로 시작 (`fetchAsisColumns` , `applyOverride`)
- 상수: SCREAMING_SNAKE_CASE
- 패키지: 단수 명사 (`mapping` , `binding` , 복수 금지)

— 20.2 TypeScript / React

- 컴포넌트: PascalCase 파일명 + default export
- 훅: `use` 접두 + camelCase (`useColumnMappings`)
- 타입: PascalCase, 인터페이스 접두 `I` 사용 안 함
- 상수: SCREAMING_SNAKE_CASE 또는 camelCase (단일 모듈 내부)

— 20.3 SQL

- 테이블·컬럼: snake_case
 - 인덱스: `ix_<table>_<columns>`
 - 외래키: `fk_<table>_<ref>`
 - 유니크: `uq_<table>_<columns>`
-

21. 부록 C. 코드 스타일

— 21.1 Java

- `.editorconfig` 에 들여쓰기 4 스페이스, LF, UTF-8 강제
- Google Java Format 적용 (Gradle 플러그인)
- import 정렬: java → javax → org → com → 기타 → 정적
- `var` 는 우변에서 타입이 명백할 때만 사용 (체이닝·제네릭 추론 X)

— 21.2 TypeScript

- ESLint + Prettier 강제, `tsconfig.json` 의 `strict: true`

- `any` 사용 금지 (불가피한 경우 `// eslint-disable-next-line ... -- 사`
유 명시)
- 컴포넌트 props 는 인라인 type alias (`type Props = { ... } )`

— 21.3 커밋 메시지

Conventional Commits 준수:

```
feat(mapping): add NULL strategy in column inspector
fix(batch): retry transient connection drop in transformStep
docs(user): clarify cutover preflight rules
refactor(api): extract ColumnOverride mapper
```

22. 부록 D. 호환성 정책

REST API 또는 도메인 모델의 비호환 변경은 릴리즈 노트와 커밋 메시지에
서 `BREAKING:` 접두로 명시하고, 클라이언트가 따라야 할 마이그레이션 절
차를 함께 제공합니다. 마이너 릴리즈에는 비호환 변경을 포함하지 않습니
다.

문서 끝.